GitHub automatisé : Projet GitBash

Présentation du projet

Le script gitbash.sh crée en une seule commande un dépôt GitHub prêt à rendre : dépôt privé, collaborateurs invités, dossiers Job01, Job02… et premier push.

À La Plateforme, chaque TP se rend sous forme de dépôt GitHub organisé en dossiers Job. Faire ces étapes à la main à chaque fois est long et source d'erreurs. Le projet consiste à écrire en Bash l'outil qui automatise cette tâche, en le sécurisant dès la conception.

Compétences travaillées

- Scripting Bash : fonctions, gestion des erreurs, formulaire interactif avec dialog
- Utilisation d'une API REST authentifiée : l'API GitHub
- Sécurisation d'un script : validation des entrées, protection du token, moindre privilège

Rendu : dépôt privé github.com/cylia-ouldamara/GitBash, créé par le script lui-même, avec l'enseignant ajouté comme collaborateur.

Prérequis et installation

Le script tourne sur une VM Debian 12 sans interface graphique, pilotée en SSH depuis le PC hôte.

Élément	Rôle
Debian 12 (sans bureau)	Système de la VM, léger et suffisant : dialog s'affiche dans le terminal
git	Créer le dépôt local, faire le commit et le push
curl	Envoyer les requêtes à l'API GitHub
jq	Lire les réponses JSON de GitHub et fabriquer le JSON envoyé
dialog	Afficher le formulaire interactif

Installation

```
sudo apt install git curl jq dialog openssh-server
git config --global user.name "Prénom Nom"
git config --global user.email "email@exemple.fr"
```

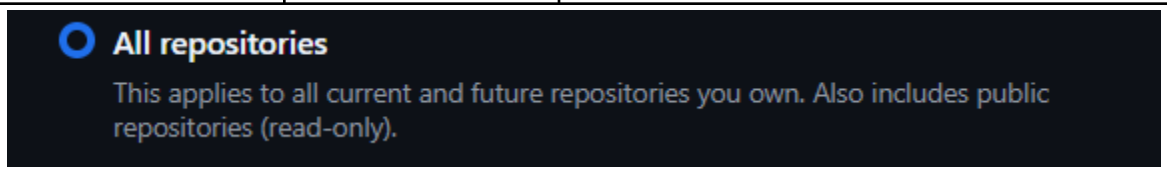
Les fichiers gitbash.sh et README.md sont ensuite copiés dans ~/projet-gitbash (avec scp ou en collant leur contenu dans nano), puis le script est rendu exécutable avec chmod +x gitbash.sh.

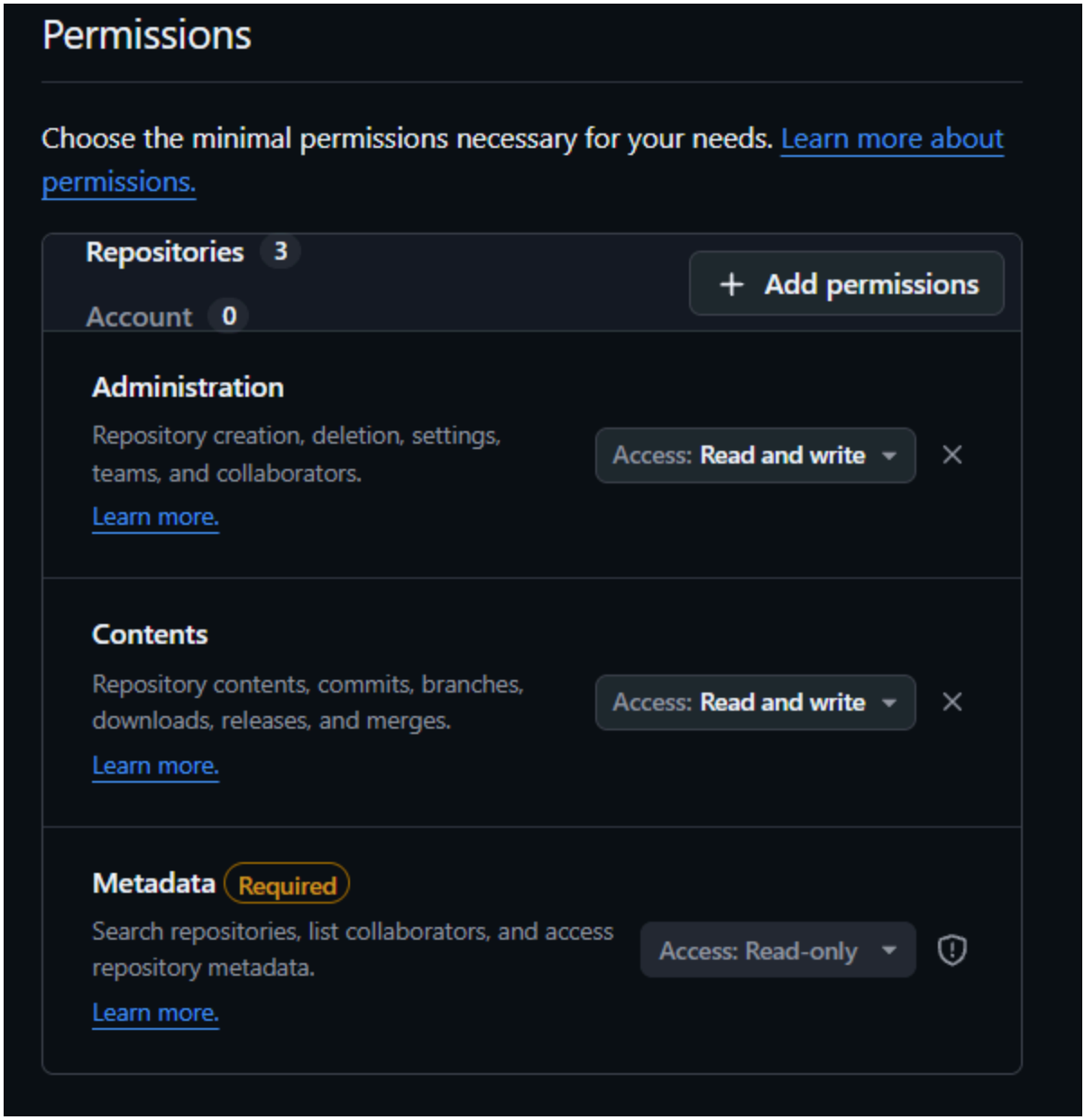
Token GitHub

Le script s'authentifie auprès de GitHub avec un token fine-grained aux droits limités, rangé dans un fichier lisible par moi seule.

Création : GitHub → Settings → Developer settings → Personal access tokens → Fine-grained tokens → Generate new token.

Réglage	Valeur	Pourquoi
Repository access	All repositories	Le dépôt n'existe pas encore au moment de créer le token
Administration	Read and write	Créer le dépôt et inviter les collaborateurs
Contents	Read and write	Faire le push
Expiration	Courte (30 jours)	Limiter les dégâts si le token fuit





Stockage dans la VM :

```
mkdir -p ~/.config/gitbash
( umask 077; read -rsp "Token : " t; printf '%s\n' "$t" > ~/.config/gitbash/token )
```

- read -s masque la saisie : le token ne s'affiche pas à l'écran.
- umask 077 crée le fichier avec les droits 600 : lecture et écriture pour moi seule.
- Le token n'est jamais tapé dans une commande, donc il n'apparaît pas dans l'historique Bash.

Utilisation

Une seule commande suffit ; le script pose ensuite ses questions dans des fenêtres dialog.

```
cd ~/projet-gitbash
./gitbash.sh
```

Fenêtre	Exemple de réponse	Règle vérifiée
Nom du dépôt	gitbash	Lettres, chiffres, . _ - uniquement
Nombre de jobs	3	Entier de 1 à 20
Collaborateurs	login-du-prof binome	Logins GitHub séparés par des espaces, lettres, chiffres et -
Droits	pull ou push	Lecture seule ou lecture + écriture
Confirmation	Yes	Récapitulatif avant toute action

Résultat à l'écran

```
✔ Dépôt créé : https://github.com/cylia-ouldamara/gitbash
✔ login-du-prof invité (pull)
✔ Dossiers Job01 à Job03 créés
✔ Premier push effectué
```

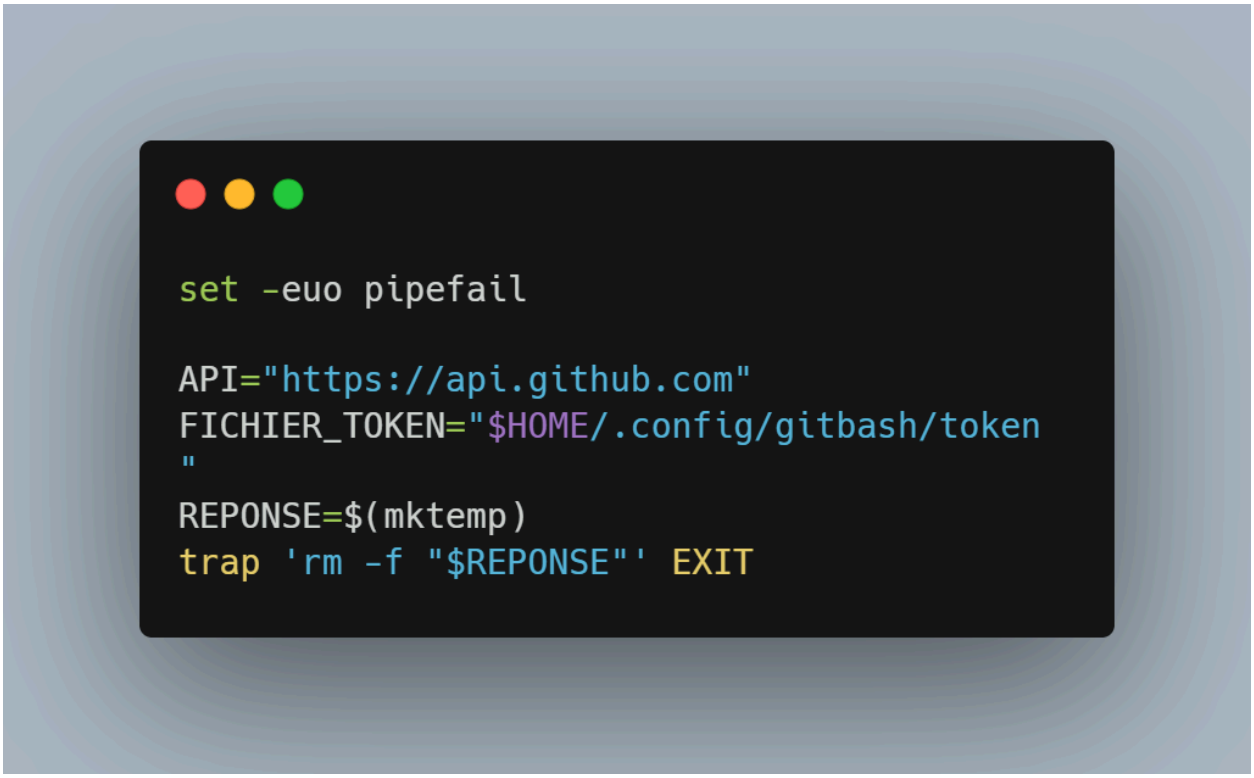
Arborescence créée

```
gitbash/
├── .gitignore
├── README.md
├── Job01/README.md
├── Job02/README.md
└── Job03/README.md
```

Fonctionnement du script

Le script se lit de haut en bas en 7 étapes ; il s'arrête dès qu'une étape échoue grâce à set -euo pipefail.

1. **Vérifications** : présence de git, curl, jq et dialog ; git config renseigné ; fichier du token présent et protégé en 600.
2. **Identification** : requête GET /user à l'API GitHub. Code 200 = token valide ; le login est extrait de la réponse avec jq.
3. **Formulaire** : fenêtres dialog pour le nom, le nombre de jobs, les collaborateurs et leurs droits. Chaque réponse est vérifiée par une expression régulière, puis un récapitulatif demande confirmation.
4. **Création du dépôt** : POST /user/repos avec {"name": …, "private": true}. Code attendu : 201.
5. **Collaborateurs** : PUT /repos/<login>/<dépôt>/collaborators/<invité> avec la permission choisie. Code 201 = invitation envoyée.
6. **Dossiers locaux** : une boucle for crée Job01 à JobNN (numérotation sur 2 chiffres avec printf "Job%02d"), chacun avec un README.md, plus un .gitignore.
7. **Commit et push** : git init, git add, git commit, ajout du remote puis git push -u origin main.



```
set -euo pipefail

API="https://api.github.com"
FICHIER_TOKEN="$HOME/.config/gitbash/token
"
REPONSE=$(mktemp)
trap 'rm -f "$REPONSE"' EXIT
```



```
erreur() {
    dialog --title "Erreur" --msgbox "$1" 8 60
    clear
    exit 1
}
```

```bash
api() {
    local donnees=()
    if [ -n "${3:-}" ]; then
        donnees=(-d "$3")
    fi
    echo "header = \"Authorization: Bearer $TOKEN\"" | curl --config - \
        --silent --proto =https \
        -X "$1" \
        -H "Accept: application/vnd.github+json" \
        -o "$REPONSE" -w "%{http_code}" \
        "${donnees[@]}" \
        "$API$2"
}
```

```bash
for outil in git curl jq dialog; do
    if ! command -v "$outil" > /dev/null; then
        echo "Il manque $outil"
        exit 1
    fi
done

if [ "$(stat -c %a "$FICHIER_TOKEN")" != "600" ]; then
    echo "Le fichier du token est mal protégé"
    exit 1
fi

TOKEN=$(cat "$FICHIER_TOKEN")
```

```
code=$(api GET /user)
if [ "$code" != "200" ]; then
    erreur "Token refusé par GitHub (code $code)"
fi
LOGIN=$(jq -r .login "$REPONSE")
```

```
NOM=$(dialog --stdout --title "Nouveau dépôt" --inputbox "Nom du dépôt :" 8 50)

if [[ ! "$NOM" =~ ^[A-Za-z0-9._-]+$ ]]; then
    erreur "Nom invalide"
fi

DROITS=$(dialog --stdout --menu "Quels droits ?" 10 50 2 \
    pull "Lecture seule (enseignant)" \
    push "Lecture et écriture (binôme)")
```

```
json=$(jq -n --arg nom "$NOM" '{name: $nom, private: true}')
code=$(api POST /user/repos "$json")
if [ "$code" != "201" ]; then
    erreur "Création impossible (code $code)"
fi
echo "✓ Dépôt créé : https://github.com/$LOGIN/$NOM"
```

```
for c in $COLLABS; do
    json=$(jq -n --arg d "$DROITS" '{permission: $d}')
    code=$(api PUT "/repos/$LOGIN/$NOM/collaborators/$c"
d$json")

for i in $(seq 1 "$NB_JOBS"); do
    job=$(printf "Job%02d" "$i")
    mkdir "$job"
    echo "# $job" > "$job/README.md"
done

printf '.env\n*.token\n*.key\ntoken\n' > .gitignore
```



```
git init -q -b main
git add .
git commit -q -m "Premier commit : création des jobs"
git remote add origin "https://github.com/$LOGIN/$NOM.git"

GIT_TOKEN="$TOKEN" git \
    -c credential.helper= \
    -c credential.helper='!f() { echo "username=x-access-token"; echo "password=$GIT_TOKEN"; }; f' \
    push -q -u origin main
```

La fonction api

Toutes les requêtes passent par une seule fonction, ce qui évite de répéter le code :

```
api() {
  # $1 = méthode, $2 = chemin, $3 = JSON (facultatif)
  echo "header = \"Authorization: Bearer $TOKEN\"" | curl --config - \
    --silent --proto =https -X "$1" \
    -H "Accept: application/vnd.github+json" \
    -o "$REPONSE" -w "%{http_code}" "${donnees[@]}" "$API$2"
}
```

Elle renvoie le code HTTP ; le corps de la réponse est rangé dans un fichier temporaire supprimé automatiquement à la fin (trap … EXIT).

Sécurité

Le token n'apparaît jamais à l'écran, dans une commande, dans .git/config ni sur GitHub, et chaque saisie est contrôlée avant d'être utilisée.

Risque	Mesure dans le script	Principe
Vol du token sur la machine	Fichier en chmod 600, vérifié au lancement	Gestion des secrets
Token visible avec ps	Passé à curl par l'entrée standard (echo … | curl --config -), pas en argument	Gestion des secrets
Token enregistré dans .git/config	Adresse du dépôt sans token ; git le lit dans une variable seulement pendant le push	Gestion des secrets
Secret envoyé sur GitHub	.gitignore qui exclut .env, *.token, *.key	Gestion des secrets

Risque	Mesure dans le script	Principe
Injection de commandes	Nom, logins et nombre vérifiés par regex (liste blanche) ; variables toujours entre guillemets	Validation des entrées
JSON malformé ou détourné	JSON fabriqué avec `jq --arg`, jamais par concaténation de texte	Validation des entrées
Interception réseau	`curl --proto =https` : HTTPS obligatoire	Communication sécurisée
Trop de droits	Dépôt privé ; enseignant en `pull` (lecture seule) ; token limité à 2 permissions et à durée courte	Moindre privilège
Erreur silencieuse	`set -euo pipefail` ; chaque code HTTP est vérifié	Gestion des erreurs
Écrasement de données	Arrêt si un dossier du même nom existe déjà	Gestion des erreurs

Exemple de validation par regex

```
if [[ ! "$NOM" =~ ^[A-Za-z0-9._-]+$ ]]; then
   erreur "Nom invalide. Utilise seulement des lettres, chiffres, . _ -"
fi
```

Une saisie comme `test; rm -rf ~` est refusée avant d'atteindre la moindre commande.

Problèmes rencontrés et solutions

Quatre blocages sont apparus pendant le premier lancement réel ; chacun a été corrigé et le script a été amélioré en conséquence.

Problème	Cause	Solution
Rien ne s'affiche en tapant le token	Saisie volontairement masquée (`read -s`)	Comportement normal : coller puis Entrée, vérifier avec `wc -c`
Fichiers absents de la VM après copie	Copie non arrivée dans le bon dossier	Recherche avec `find ~ -name gitbash.sh`, copie via SSH
« Identité d'auteur inconnue » au commit	`git config user.name` / `user.email` jamais configurés	Configuration de git ; ajout d'une vérification au début du script
Erreur 403 « Write access not granted » au push	Token sans la permission Contents en écriture	Modification du token sur GitHub (Contents : Read and write)

Conclusion et améliorations possibles

Le script crée un dépôt de TP complet en moins d'une minute, sans exposer le token, et a servi à générer le dépôt de rendu lui-même.

Ce projet m'a appris à structurer un script Bash en étapes, à dialoguer avec une API REST via curl et jq, et surtout à penser sécurité dès le départ : un secret ne doit jamais apparaître dans une commande, un fichier versionné ou un log.

Améliorations possibles

- Ajouter des options en ligne de commande (-n, -j…) pour lancer le script sans formulaire.
- Proposer de réutiliser un dépôt existant au lieu de s'arrêter.
- Ajouter des jobs à un dépôt déjà créé (Job04, Job05…).
- Tenir un journal des actions dans un fichier de log, sans le token.